**Queen Mary University of London, UK**

**MSc Computer Science**

# London Underground Transportation Database System

**YASIN YALCIN**

**Table of Contents**


*PART 1: DATABASE DESIGN* ..... *2*

*1. Conceptual Schema and Assumptions* ..... *2*

**1.1 Entity Descriptions and Design Assumptions** ..... **2**

**1.1.1. PassengerCard** ..... 2

**1.1.2. OysterCard** ..... 2

**1.1.3. BankCard** ..... 3

**1.1.4. Ticket** ..... 3

**1.1.5. CheckIn** ..... 3

**1.1.6. CheckOut** ..... 4

**1.1.7. Station** ..... 4

**1.1.8. Line** ..... 4

**1.1.9. Holiday** ..... 5

**1.2 Additional Design Assumptions** ..... **5**

*2. Relational Schema* ..... *6*

**2.1. Strong Entities** ..... **6**

**2.2. Specialisation (Subclass Entities)** ..... **6**

**2.3. One-to-Many (1:N) Relationships** ..... **6**

**2.4. Many-to-Many (M:N) Relationships** ..... **7**

**2.5. Final Relational Schema** ..... **7**

*3. Normalisation* ..... *8*

**3.1 Universal Relation** ..... **8**

**3.2 Functional Dependencies** ..... **8**

**3.3 First Normal Form (1NF)** ..... **8**

**3.4 Second Normal Form (2NF)** ..... **8**

**3.5 Third Normal Form (3NF)** ..... **8**

**3.6 BCNF Verification** ..... **9**

**3.7 Final Normalised Design** ..... **9**

*PART 2: DATABASE IMPLEMENTATION* ..... *10*

*4. Implementation Components* ..... *10*

**4.1 CREATE TABLE Commands** ..... **10**

**4.2 TRIGGERS** ..... **10**

**4.3 Sample Test Data** ..... **10**

**4.4 Views** ..... **11**

**4.5 Queries** ..... **13**

**PART 1: DATABASE DESIGN**

**1. Conceptual Schema and Assumptions**

The primary purpose of the London Underground Transportation Database System is to record, monitor, and analyse how passengers use different travel card types on the city's public transport network and their travel patterns. The database ensures accurate, organised, and up-to-date data recording of passenger movements so it can be used for analysis and decision-making.

In detail, the database includes the following information:

- Descriptions of various travel card types such as Oyster, Bank Card, or Printed Ticket, together with their balance or expiry information.
- Passenger travel activity — check-ins and check-outs at stations are logged along with the time, fare, and line.
- Transport infrastructure details: the stations, the lines, and whether barrier systems are present at stations.
- Dates such as public holidays, which can influence passenger behaviour and fare patterns.

Based on this design, the core entities identified as the foundation of the database to support the system's main objectives are **PassengerCard, OysterCard, BankCard, Ticket, CheckIn, CheckOut, Station, Line, Holiday,** and the relationships among them.

**1.1 Entity Descriptions and Design Assumptions**

This section describes the entities within the London Underground Transportation Database System in detail, including their attributes, assumptions, and relationships with other entities in the model.

**1.1.1. PassengerCard**

**Attributes:**

CardID (PK), CardType

**Assumptions and Descriptions:**

1. CardID serves as the primary key and uniquely identifies each card.
2. CardType can be Oyster, Bank Card, or Ticket.
3. This is a superclass entity that holds the common attributes shared across all card types.
4. A passenger may own multiple cards. For instance, a passenger might have both an Oyster Card and a Bank Card.

**Relationships:**

- One-to-many with CheckIn: A single PassengerCard may be linked to several check-ins (PassengerCard side, partial participation; CheckIn side, total participation).
- One-to-many with CheckOut: One PassengerCard can be related to many check-outs (PassengerCard side, partial participation; CheckOut side, total participation).
- Specialisation with OysterCard, BankCard, Ticket: This is a disjoint and total Specialisation; every PassengerCard must belong to exactly one subtype (total participation) and cannot belong to more than one subtype (disjoint).

**1.1.2. OysterCard**

**Attributes:**

CardID (PK, FK), CardHolderName, CreditAmount

**Assumptions and Descriptions:**

1. This entity extends PassengerCard, storing additional card-specific details for Oyster cards.
2. CreditAmount reflects the current balance on the card.
3. CardHolderName may be NULL - some Oyster cards are anonymous.

**Relationships:**

- Subclass of PassengerCard: OysterCard participates totally (every OysterCard is a PassengerCard) and the Specialisation is disjoint.

#### 1.1.3. BankCard

**Attributes:**

CardID (PK, FK), CardHolderName, ExpirationDate

**Assumptions and Descriptions:**

1. This entity extends PassengerCard, storing additional card-specific details for Bank cards.
2. CardHolderName is required (NOT NULL) as all bank cards are associated with a cardholder's name.
3. ExpirationDate stores the date the card expires.
4. Real card numbers should not be used in test data for reasons of privacy and security.

**Relationships:**

- Subclass of PassengerCard: BankCard participates totally under the PassengerCard supertype and belongs to a disjoint Specialisation.

#### 1.1.4. Ticket

**Attributes:**

CardID (PK, FK), ValidUntil

**Assumptions and Descriptions:**

1. This entity extends PassengerCard, storing additional card-specific details for printed tickets.
2. ValidUntil indicates when the ticket expires.
3. Tickets are usually anonymous with no cardholder name.

**Relationships:**

- Subclass of PassengerCard: Ticket participates fully under the PassengerCard supertype and belongs to a disjoint Specialisation.

#### 1.1.5. CheckIn

**Attributes:**

CheckInID (PK), CheckInTime, CheckInType

**Assumptions and Descriptions:**

1. CheckInID uniquely identifies each check-in event.
2. CheckInTime records the timestamp of when the check-in occurred.
3. CheckInType differentiates between regular check-ins and pink card reader (transit) check-ins.
4. Any check-in should point to a card, station, and line.

**Relationships:**

- Many-to-one with PassengerCard: Each CheckIn is made by exactly one PassengerCard, total participation on CheckIn side, partial participation on PassengerCard side.
- Many-to-one with Station: Each CheckIn occurs at one and only one Station (total participation on CheckIn side, partial participation on Station side)
- Many-to-one with Line: Each CheckIn is done on precisely one Line; total participation on the CheckIn side, partial participation on the Line side

#### 1.1.6. CheckOut

**Attributes:**

CheckOutID (PK), CheckOutTime, FareAmount, PreviousCredit, NextCredit

**Assumptions and Descriptions:**

1. CheckOutID uniquely identifies each check-out event.
2. CheckOutTime records the timestamp of when the check-out occurred.
3. FareAmount stores the fare charged for this journey.
4. PreviousCredit and NextCredit record the balance on the Oyster card before and after the transaction (NULL for other card types).
5. All check-outs are to be associated with a card, station, and line.
6. Check-outs can optionally happen on a holiday.

**Relationships:**

- Many-to-one with PassengerCard: Each CheckOut is made by exactly one PassengerCard, total participation on CheckOut side, partial participation on PassengerCard side.
- Many-to-one with Station: Each CheckOut occurs at exactly one Station. Total participation on the CheckOut side, partial participation on the Station side.
- Many-to-one with Line: Each CheckOut occurs on exactly one Line (total participation on CheckOut side, partial participation on Line side).
- Association with Holiday: As shown in the conceptual schema, each CheckOut may be associated with zero or one Holiday. In fact, this association exists between a check-out date matching the date of a recorded public holiday.

#### 1.1.7. Station

**Attributes:**

StationID (PK), StationName, HasBarrier, Zone

**Assumptions and Descriptions:**

1. StationID is the identifier for each unique station.
2. StationName contains the name of the station.
3. HasBarrier: A boolean value indicating whether the station is equipped with entry/exit barriers.
4. Zone demonstrates the transport area, such as Zone 1 and Zone 6.

**Relationships:**

- One-to-many with CheckIn: Each Station might be linked to multiple check-ins (Station side partial participation, CheckIn side total participation).
- One-to-many with CheckOut: Every station is allowed to have numerous check-outs (Station side partial participation, CheckOut side total participation).
- Many-to-many with Line via StationLine associative entity (partial participation on both sides).

#### 1.1.8. Line

**Attributes:**

LineID (PK), LineName

**Assumptions:**

1. LineID is the unique value used to identify each line.
2. LineName indicates the name of the line, such as “Central Line” or “DLR.”

**Relationships:**

- One-to-many with CheckIn: Each Line can have multiple check-ins (partial participation on Line side, total participation on CheckIn side).
- One-to-many with CheckOut: Each Line can have multiple check-outs - partial participation on Line's side and total participation on CheckOut's side.
- Many-to-many with Station through associative entity StationLine (partial participation on both sides).

### 1.1.9. Holiday

**Attributes:**

HolidayDate (PK), HolidayName

**Assumptions and Descriptions:**

1. HolidayDate is the unique identifier for any given holiday.
2. HolidayName is the name of the holiday, for instance, "Christmas Day", "New Year's Day".

**Relationships:**

- **Association with CheckOut:** A Holiday can be associated with multiple CheckOut events occurring on that date. This connection indicates the presence of several check-outs on the particular day that the holiday represents.

### 1.2 Additional Design Assumptions

- **Check-In/Check-Out Matching:** Check-In and Check-Out tables are physically separated to record incomplete journeys — that is, missing tap-in or tap-out. Matching is carried out during query time by pairing entries with the same CardID and logically consecutive timestamps.
- **Fare Calculation:** The FareAmount is set by an external transport system and stored as provided. The database system does not keep any information on the fare calculation process.
- **Credit Tracking:** PreviousCredit and NextCredit are stored in CheckOut to keep a historical record of balance changes per transaction, rather than storing only the current balance in OysterCard.

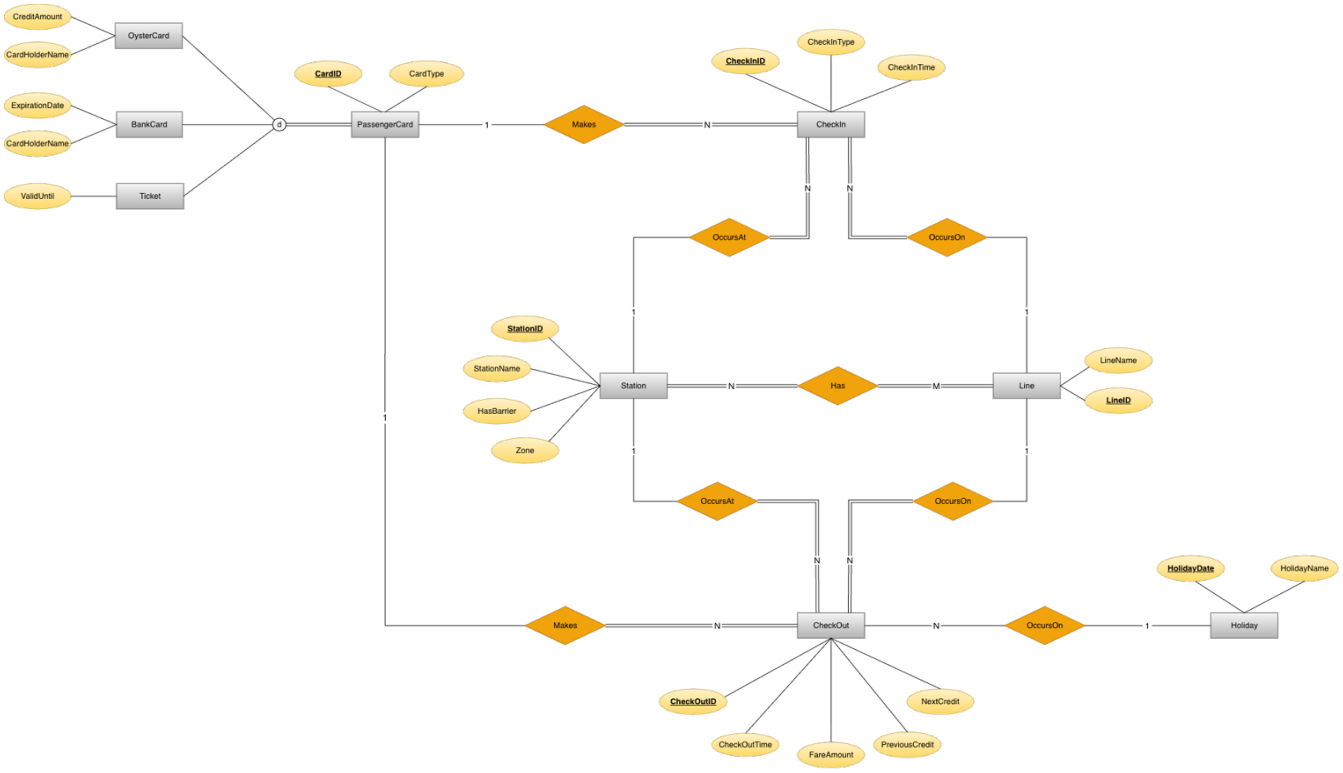



**Figure 1: Conceptual Schema**

## 2. Relational Schema

The relational model was built by taking the elements of the conceptual design and reshaping them using established ER-to-relational conversion techniques. This ensured that the original entities, their relations, and the relevant constraints were carried over correctly. Each component of the schema was then organised into its own table, keeping the links between tables consistent and allowing passenger journeys and transport operations to be examined in a structured way.

### 2.1. Strong Entities

Each independent entity from the conceptual model is represented as a separate relation. Their attributes are directly mapped into columns, with primary keys for unique identification of records. The strong entities in the design are:

- **PassengerCard**, acting as the superclass for all card types.
- **Station**, storing physical and operational characteristics of stations.
- **Line**, representing transport routes inside the network.
- **Holiday,** including special dates that might affect fare behaviour.
- **CheckIn** and **CheckOut**, transactional entities representing the events of passenger movement.

These relations form the structural core of the database, ensuring that essential passenger, infrastructure, and operational data is stored without redundancy.

### 2.2. Specialisation (Subclass Entities)

The system implements a supertype–subtype structure centred on PassengerCard. PassengerCard is the supertype, while:

- **OysterCard**
- **BankCard**
- **Ticket**

act as subtypes, each with attributes that concern only their respective card type.

This Specialisation is:

- **Total participation** – every PassengerCard must belong to exactly one subtype.
- **Disjoint**, meaning no card can have more than one subtype.
- **Implemented via shared primary keys** – each subtype inherits CardID as both its PK and FK.

This approach prevents unnecessary NULL values and ensures that attributes such as CreditAmount (OysterCard) or ExpirationDate (BankCard) are stored only where appropriate.

### 2.3. One-to-Many (1:N) Relationships

Several relationships in the model are one-to-many. In each case, the "many" side stores a foreign key referencing the "one" side:

- **CheckIn** stores **CardID**, **StationID**, and **LineID**, thereby interrelating the check-in event with a card, station, and line.
- **CheckOut** maintains the same foreign keys: **CardID**, **StationID**, **LineID**. Note: The Holiday association is derived by matching the date portion of CheckOutTime with HolidayDate, so no foreign key is stored. A **station** or **line** can appear in many **CheckIn** or **CheckOut** records, while each event of movement must relate to exactly one station and one line.

These relations are the central operational elements of the system, associating passengers with locations and times in a regular structure.

### 2.4. Many-to-Many (M:N) Relationships

- The Station and Line entities represent a many-to-many association in that one line may contain many stations and one station may belong to many lines.
- To implement the association between the two relations, an association relation named StationLine is used. This relation includes StationID and LineID in its composite primary key. This ensures the association's uniqueness.
- This makes the process of routing easy by allowing direct queries on various transport routes.

### 2.5. Final Relational Schema

The final relational schema, with primary and foreign key definitions, is presented below:

Primary keys (PK) are bold and underlined, and foreign keys are indicated as (FK).

PassengerCard (**CardID (PK),** CardType)
OysterCard (**CardID (PK, FK),** CardHolderName, CreditAmount)
BankCard (**CardID (PK, FK),** CardHolderName, ExpirationDate)
Ticket (**CardID (PK, FK),** ValidUntil)
CheckIn (**CheckInID (PK),** CardID (FK), StationID (FK), LineID (FK), CheckInTime, CheckInType)
CheckOut (**CheckOutID (PK),** CardID (FK), StationID (FK), LineID (FK), CheckOutTime, FareAmount, PreviousCredit, NextCredit)
Station (**StationID (PK),** StationName, HasBarrier, Zone)
Line (**LineID (PK),** LineName)
StationLine (**StationID (PK, FK)**, **LineID (PK, FK)**)
Holiday (**HolidayDate (PK)**, HolidayName)

This relational schema ensures data consistency through strong referential integrity constraints and normalisation. The design reduces redundancy and allows flexibility for analysing travel behaviour, monitoring passenger flows, and supporting future extensions.

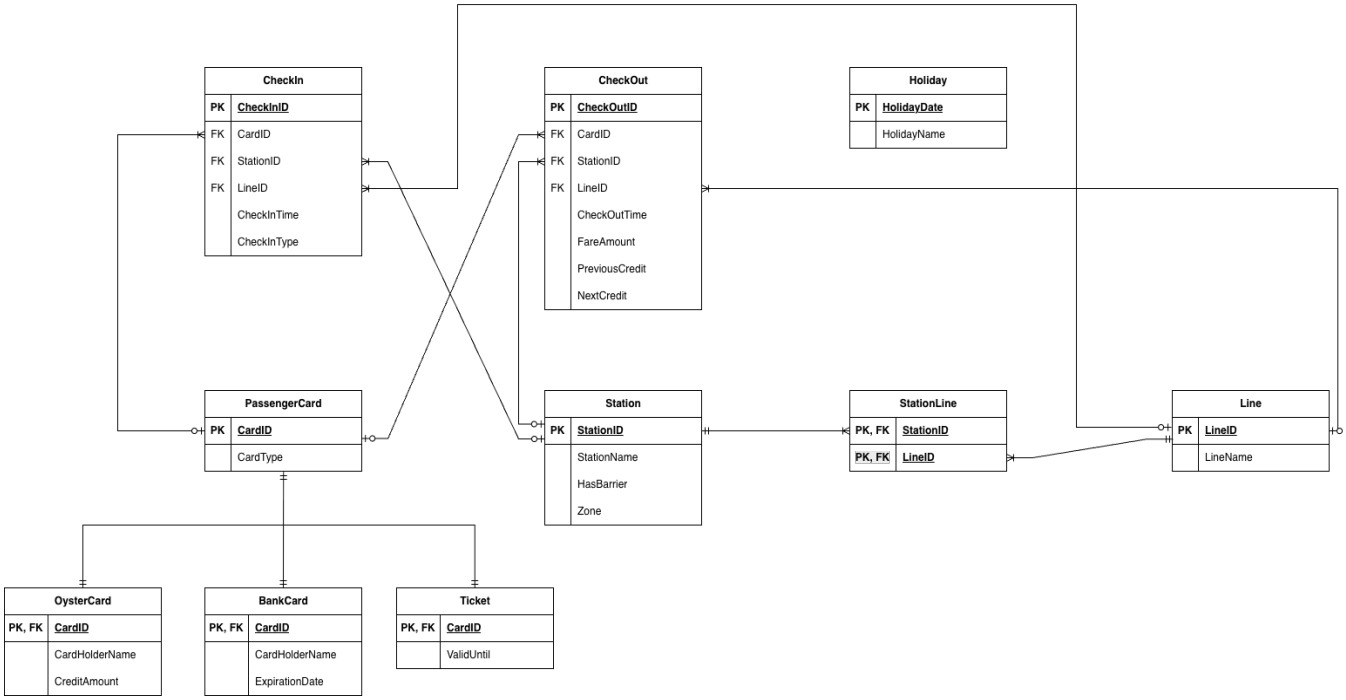



**Figure 2: Relational Schema**

## 3. Normalisation

### 3.1 Universal Relation

Normalisation was performed to remove redundancy, maintain data integrity, and make sure the London Underground Transportation Database System can support the analysis process. Conceptual design is first represented by expressing the entire relation consisting of all the attributes from the entities and their relations.

This universal relation U is defined as:

U (CardID, CardType, CardHolderName, CreditAmount, ExpirationDate, ValidUntil, CheckInID, CheckInTime, CheckInType, CheckOutID, CheckOutTime, FareAmount, PreviousCredit, NextCredit, StationID, StationName, HasBarrier, Zone, LineID, LineName, HolidayDate, HolidayName)

Each tuple in the relation corresponds to a passenger flow record. It includes the passenger's card details along with check-in and check-out information. Such data is closely related to the transport infrastructure.

### 3.2 Functional Dependencies

Based on the conceptual schema, the following functional dependencies have been identified:

- CardID → CardType, CardHolderName, CreditAmount, ExpirationDate, ValidUntil
- CheckInID → CheckInTime, CheckInType, CardID, StationID, LineID
- CheckOutID → CheckOutTime, FareAmount, PreviousCredit, NextCredit, CardID, StationID, LineID
- StationID → StationName, HasBarrier, Zone
- LineID → LineName
- HolidayDate → HolidayName
- {StationID, LineID} → (no additional attributes; this represents the many-to-many association)

The final dependency corresponds to the many-to-many association rule between stations and lines. This association will be settled in the **StationLine** relation.

### 3.3 First Normal Form (1NF)

The universal relation satisfies First Normal Form (1NF) because all its fields contain atomic values. There are no repeating groups. Each field in the relation represents one data element. Thus, all relations are guaranteed to contain unique values identifiable through their key attributes.

### 3.4 Second Normal Form (2NF)

To achieve Second Normal Form (2NF), the partial dependencies were eliminated. The universal relation contained attributes determined by different keys. To achieve 2NF, these attributes were separated into distinct relations based on their respective determinants. Hence, the attributes would be divided into new relations depending on the appropriate part of the key. Thus, the 2NF relations are given by:

- PassengerCard (**CardID (PK),** CardType, CardHolderName, CreditAmount, ExpirationDate, ValidUntil)
- CheckIn (**CheckInID (PK),** CardID (FK), StationID (FK), LineID (FK), CheckInTime, CheckInType)
- CheckOut (**CheckOutID (PK),** CardID (FK), StationID (FK), LineID (FK), CheckOutTime, FareAmount, PreviousCredit, NextCredit)
- Station (**StationID (PK),** StationName, HasBarrier, Zone)
- Line (**LineID (PK),** LineName)
- StationLine (**StationID (PK, FK), LineID (PK, FK)**)
- Holiday (**HolidayDate (PK),** HolidayName)

The StationLine table resolves the many-to-many relation between Station and Line entities through the use of a composite primary key (StationID, LineID), along with two foreign keys pointing back to the respective parent tables to maintain referential integrity.

### 3.5 Third Normal Form (3NF)

A relation is in Third Normal Form (3NF) if it satisfies 2NF and has no transitive dependencies, meaning non-prime attributes depend only on the candidate key. According to the condition for 3NF, the PassengerCard table is not in 3NF because some attributes depend on the CardType field instead of on the primary key CardID:

- CreditAmount applies only when CardType is 'Oyster'
- ExpirationDate applies only when CardType is 'BankCard'
- ValidUntil applies only when CardType is 'Ticket'
- CardHolderName applies to OysterCard and BankCard, but not to Ticket

This is a transitive dependency wherein the attributes depend on CardID through CardType. Thus, the attributes that depend on the type of cards were split into specific relations. This resulted in the generation of the relations below:

- PassengerCard (**CardID (PK),** CardType)
- OysterCard (**CardID (PK, FK),** CardHolderName, CreditAmount)
- BankCard (**CardID (PK, FK),** CardHolderName, ExpirationDate)
- Ticket (**CardID (PK, FK),** ValidUntil)

It guarantees that every subtype of the cards holds only the respective attributes of the subtype, while the CardID is used simultaneously as the primary key and foreign key in the subtype relations. This is a disjoint and total Specialisation, meaning each PassengerCard belongs to exactly one subtype.

The other 2NF relations (CheckIn, CheckOut, Station, Line, StationLine, Holiday) do not possess transitive dependencies. Every non-key field in the relations directly relies on the primary key.

**Refinement of the Holiday Relationship:** The Conceptual Schema (Figure 1) shows an association between CheckOut and Holiday. However, adding HolidayDate as a foreign key in the CheckOut table would store the same date value twice, once in CheckOutTime and once in HolidayDate. This duplication violates normalisation principles. Therefore, the foreign key was omitted. Instead, the relationship is derived in queries by matching the date portion of CheckOutTime with HolidayDate.

### 3.6 BCNF Verification

To further validate the process of normalisation, each relation was evaluated for Boyce-Codd Normal Form (BCNF). BCNF mandates the elimination of all redundancy because of functional dependencies. Non-key values in BCNF depend on the primary key.

The following determinants exist in the relations:

- CardID → CardType (in PassengerCard)
- CardID → CardHolderName, CreditAmount (in OysterCard)
- CardID → CardHolderName, ExpirationDate (in BankCard)
- CardID → ValidUntil (in Ticket)
- CheckInID → CheckInTime, CheckInType, CardID, StationID, LineID (in CheckIn)
- CheckOutID → CheckOutTime, FareAmount, PreviousCredit, NextCredit, CardID, StationID, LineID (in CheckOut)
- StationID → StationName, HasBarrier, Zone (in Station)
- LineID → LineName (in Line)
- HolidayDate → HolidayName (in Holiday)

Every determinant in each relation is a candidate key. Therefore, the schema satisfies BCNF and no further decomposition is needed.

### 3.7 Final Normalised Design

The final database design satisfies Third Normal Form (3NF) and complies with Boyce-Codd Normal Form (BCNF). The decomposition ensures data consistency, removes redundancy, and maintains referential integrity across all entities. This schema provides a solid foundation for analysing passenger movements, identifying issues, and supporting real-time monitoring within the London Underground Transportation Database System.

**The final normalised schema comprises:**

- PassengerCard (**CardID (PK),** CardType)
- OysterCard (**CardID (PK, FK),** CardHolderName, CreditAmount)
- BankCard (**CardID (PK, FK),** CardHolderName, ExpirationDate)
- Ticket (**CardID (PK, FK),** ValidUntil)
- CheckIn (**CheckInID (PK),** CardID (FK), StationID (FK), LineID (FK), CheckInTime, CheckInType)
- CheckOut (**CheckOutID (PK),** CardID (FK), StationID (FK), LineID (FK), CheckOutTime, FareAmount, PreviousCredit, NextCredit)
- Station (**StationID (PK),** StationName, HasBarrier, Zone)
- Line (**LineID (PK),** LineName)
- StationLine (**StationID (PK, FK), LineID (PK, FK)**)
- Holiday (**HolidayDate (PK),** HolidayName)

Overall, the final normalised schema ensures the database design is efficient, scalable, and aligned with the logical requirements of the London Underground Transportation Database System.

## PART 2: DATABASE IMPLEMENTATION

### 4. Implementation Components

All components of the implementation process for CREATE TABLE statements, TRIGGERS, INSERT statements, VIEWS, and QUERIES are included in SQL script file named london_underground.sql.

### 4.1 CREATE TABLE Commands

The database is built from ten tables, each derived from the normalised design presented in Part 1. These tables enforce data integrity with primary and foreign keys, along with NOT NULL and CHECK constraints.

### 4.2 TRIGGERS

Declarative constraints by their nature cannot enforce certain business rules. For instance, checking the expiry date of the cards involves comparisons of dates in different tables. Also, the process of updating the Oyster Card's balance after checkout operations involves coordinated updates from the CheckOut table and the OysterCard table. To handle these scenarios, triggers have been used.

**Trigger 1: validate_travel_expiry**

This trigger prevents the entry of passengers if their BankCard or Ticket is expired. It runs prior to the insertion of the CheckIn record. This trigger checks the type of cards being used. It checks the ExpirationDate if the cards being used are BankCards. However, for tickets, it checks the ValidUntil date. When the cards/tickets are expired, the trigger raises an error and the check-in is rejected.

**Trigger 2: prevent_negative_entry**

This trigger prevents Oyster cards from entering the system if their credit amount is £0 or below. It runs prior to the insertion of the CheckIn record. The trigger detects Oyster cards by checking the CreditAmount. If the credit amount is £0 or below, the system generates an error message that prevents the entry process.

**Trigger 3: update_card_balance**

This trigger automatically updates the Oyster card balance based on the successful completion of the journey. This trigger runs immediately after the insertion of the CheckOut record. Once the checkout process contains the Oyster card (that is, NextCredit holds some value), the trigger updates the CreditAmount in the OysterCard table based on the new Oyster card balance.

### 4.3 Sample Test Data

The test data is intended to simulate realistic scenarios in the London Underground network by covering the following:

- The dataset includes 14 stations distributed across London's 6 transport zones.
- It covers 8 different transport lines: District, Jubilee, Northern, DLR, Elizabeth, Piccadilly, Central and Victoria.
- There are 15 passenger cards in total, representing three card types: 6 Oyster Cards, 5 Bank Cards and 4 Tickets.
- 45 check-in and 40 check-out entries detailing passenger movements in the network

The test data includes some anomalous values to verify the correctness of queries and views. Five incomplete journeys (check-ins without check-outs) and three check-outs without check-ins are used for the evaluation. Such cases are very crucial to check the system's ability to recognise unusual instances.

**4.4 Views**

Four views were created to support data analysis and operational monitoring.

**View 1: View total fares collected for each line.**

This view shows the total fares and the number of journeys for each transport line, making it easier to compare revenue across lines.

```
CREATE OR REPLACE VIEW LineFaresSummary AS
SELECT l.LineID, l.LineName, COUNT(*) AS JourneyCount, SUM(co.FareAmount) AS
TotalFares
FROM CheckOut co
JOIN Line l ON co.LineID = l.LineID
GROUP BY l.LineID, l.LineName
ORDER BY TotalFares DESC;
```

**Output:**

| LINEID | LINENAME | JOURNEYCOUNT | TOTALFARES |
|---|---|---|---|
| L02 | District | 12 | 46 |
| L03 | Jubilee | 6 | 23.3 |
| L04 | Northern | 4 | 22.7 |
| L05 | DLR | 6 | 22.4 |
| L06 | Elizabeth | 5 | 15.5 |
| L07 | Piccadilly | 3 | 15.3 |
| L01 | Central | 4 | 13.5 |

**View 2: View zone-based revenue summary with total check-out.**

This view provides a summary of revenue grouped by transport zone. It reports how many checkouts took place in each zone, the total revenue generated there, and the average fare for the journeys recorded.

```
CREATE OR REPLACE VIEW ZoneRevenueSummary AS
SELECT s.Zone, COUNT(co.CheckOutID) AS TotalCheckOuts, SUM(co.FareAmount) AS
TotalRevenue, ROUND(AVG(co.FareAmount), 2) AS AverageFare
FROM Station s
LEFT JOIN CheckOut co ON s.StationID = co.StationID
GROUP BY s.Zone
ORDER BY s.Zone;
```

**Output:**

| ZONE | TOTALCHECKOUTS | TOTALREVENUE | AVERAGEFARE |
|---|---|---|---|
| 1 | 19 | 86.3 | 4.54 |
| 2 | 13 | 42.1 | 3.24 |
| 3 | 5 | 14.9 | 2.98 |
| 4 | 1 | 4.7 | 4.7 |
| 5 | 1 | 5.1 | 5.1 |
| 6 | 1 | 5.6 | 5.6 |

**View 3: View incomplete journeys where passengers checked in but never checked out.**

This view identifies all journeys where a passenger checked in but never checked out. It is helpful for finding possible system issues or situations where someone may have forgotten to check-out.

```
CREATE OR REPLACE VIEW IncompleteJourneys AS
SELECT ci.CheckInID, ci.CardID, pc.CardType, ci.StationID, s.StationName,
ci.LineID, l.LineName, ci.CheckInTime, ci.CheckInType
FROM CheckIn ci
JOIN PassengerCard pc ON ci.CardID = pc.CardID
JOIN Station s ON ci.StationID = s.StationID
JOIN Line l ON ci.LineID = l.LineID
WHERE NOT EXISTS (SELECT *
                  FROM CheckOut co
                  WHERE co.CardID = ci.CardID
                  AND co.CheckOutTime > ci.CheckInTime);
```

**Output:**

| CHECKINID | CARDID | CARDTYPE | STATIONID | STATIONNAME | LINEID | LINENAME | CHECKINTIME | CHECKINTYPE |
|---|---|---|---|---|---|---|---|---|
| CI037 | OYS002 | Oyster | S10 | Westminster | L03 | Jubilee | 6/5/2025, 10:22:00 AM | Standard |
| CI040 | TKT004 | Ticket | S03 | Stratford | L03 | Jubilee | 10/21/2025, 10:41:00 AM | Standard |
| CI038 | 4539777788889999 | BankCard | S11 | Greenwich | L05 | DLR | 6/6/2025, 9:07:00 AM | Standard |
| CI039 | OYS003 | Oyster | S12 | Hammersmith | L07 | Piccadilly | 6/6/2025, 2:19:00 PM | Standard |

**View 4: View station activity with total check-in and check-out counts.**

This view counts all check-ins and check-outs at each station, making it easy to see which stations are the busiest.

```
CREATE OR REPLACE VIEW StationActivity AS
SELECT s.StationID, s.StationName, s.Zone,
       CASE WHEN s.HasBarrier = 'TRUE' THEN 'Yes' ELSE 'No' END AS HasBarrier,
       COUNT(DISTINCT ci.CheckInID) AS TotalCheckIns,
       COUNT(DISTINCT co.CheckOutID) AS TotalCheckOuts
FROM Station s
LEFT JOIN CheckIn ci ON s.StationID = ci.StationID
LEFT JOIN CheckOut co ON s.StationID = co.StationID
GROUP BY s.StationID, s.StationName, s.Zone, s.HasBarrier
ORDER BY COUNT(DISTINCT ci.CheckInID) DESC;
```

**Output:**

| STATIONID | STATIONNAME | ZONE | HASBARRIER | TOTALCHECKINS | TOTALCHECKOUTS |
|---|---|---|---|---|---|
| S01 | King's Cross St Pancras | 1 | Yes | 7 | 5 |
| S09 | Liverpool Street | 1 | Yes | 5 | 2 |
| S03 | Stratford | 2 | Yes | 5 | 3 |
| S02 | Paddington | 1 | Yes | 5 | 2 |
| S08 | Bank | 1 | Yes | 4 | 5 |
| S04 | Canary Wharf | 2 | Yes | 3 | 5 |
| S10 | Westminster | 1 | Yes | 3 | 5 |
| S12 | Hammersmith | 2 | Yes | 3 | 2 |
| S05 | Richmond | 4 | No | 2 | 1 |
| S07 | Ealing Broadway | 3 | Yes | 2 | 2 |
| S11 | Greenwich | 2 | No | 2 | 3 |
| S06 | Wimbledon | 3 | Yes | 2 | 3 |
| S13 | Stanmore | 5 | Yes | 1 | 1 |
| S14 | Heathrow Terminal 5 | 6 | Yes | 1 | 1 |

### 4.5 Queries

12 SQL queries were developed to support various analytical and operational needs.

**Query 1: Which stations are on the Northern line?**

This query lists all stations on the Northern line, sorted by zone and name.

```
SELECT s.StationID, s.StationName, s.Zone
FROM Station s
JOIN StationLine sl ON s.StationID = sl.StationID
JOIN Line l ON sl.LineID = l.LineID
WHERE l.LineName = 'Northern'
ORDER BY s.Zone, s.StationName;
```

**Output:**

| STATIONID | STATIONNAME | ZONE |
|---|---|---|
| S08 | Bank | 1 |
| S01 | King's Cross St Pancras | 1 |

**Query 2: Which bank cards and tickets have expired?**

This query finds expired bank cards and tickets. It compares expiration dates with the current date.

```
SELECT CardID, CardHolderName, ExpirationDate, 'BankCard' AS CardType
FROM BankCard
WHERE ExpirationDate < SYSDATE
UNION
SELECT CardID, '-', ValidUntil, 'Ticket' AS CardType
FROM Ticket
WHERE ValidUntil < SYSDATE;
```

**Output:**

| CARDID | CARDHOLDERNAME | EXPIRATIONDATE | CARDTYPE |
|---|---|---|---|
| 4539087654321012 | Bob Johnson | 1/31/2025, 12:00:00 AM | BankCard |
| TKT001 | - | 10/16/2024, 12:00:00 AM | Ticket |
| TKT002 | - | 12/26/2024, 12:00:00 AM | Ticket |
| TKT003 | - | 3/13/2025, 12:00:00 AM | Ticket |
| TKT004 | - | 10/23/2025, 12:00:00 AM | Ticket |

**Query 3: How much fare was collected on each line?**

This query shows how much fare was collected on each line by adding up all the fares from completed journeys.

```
SELECT l.LineID, l.LineName, SUM(co.FareAmount) AS TotalFares
FROM CheckOut co
JOIN Line l ON co.LineID = l.LineID
GROUP BY l.LineID, l.LineName
ORDER BY TotalFares DESC;
```

**Output:**

| LINEID | LINENAME | TOTALFARES |
|---|---|---|
| L02 | District | 46 |
| L03 | Jubilee | 23.3 |
| L04 | Northern | 22.7 |
| L05 | DLR | 22.4 |
| L06 | Elizabeth | 15.5 |
| L07 | Piccadilly | 15.3 |
| L01 | Central | 13.5 |

**Query 4: What is the average fare for each card type?**

This query calculates average fare by card type, showing differences between Oyster, BankCard, and Ticket users.

```
SELECT pc.CardType, COUNT(*) AS JourneyCount, ROUND(AVG(co.FareAmount), 2) AS
AverageFare
FROM CheckOut co
JOIN PassengerCard pc ON co.CardID = pc.CardID
GROUP BY pc.CardType
ORDER BY AverageFare DESC;
```

**Output:**

| CARDTYPE | JOURNEYCOUNT | AVERAGEFARE |
|---|---|---|
| Oyster | 27 | 4.09 |
| BankCard | 10 | 3.85 |
| Ticket | 3 | 3.23 |

**Query 5: How many journeys were made on each public holiday?**

This query calculates the number of journeys taken on public holidays. It helps understand travel habits during holidays.

```
SELECT TO_CHAR(h.HolidayDate, 'DD-MON-YYYY') AS HolidayDate, h.HolidayName,
COUNT(*) AS JourneyCount
FROM Holiday h
JOIN CheckOut co ON TRUNC(co.CheckOutTime) = h.HolidayDate
GROUP BY h.HolidayDate, h.HolidayName
ORDER BY h.HolidayDate;
```

**Output:**

| HOLIDAYDATE | HOLIDAYNAME | JOURNEYCOUNT |
|---|---|---|
| 26-AUG-2024 | Summer Bank Holiday | 3 |
| 25-DEC-2024 | Christmas Day | 4 |
| 26-DEC-2024 | Boxing Day | 3 |
| 01-JAN-2025 | New Year's Day | 3 |

**Query 6: How many check-ins happen in each hour of the day?**

This query displays the number of check-ins by time of day. It is used to find peak hours.

```
SELECT TO_CHAR(CheckInTime, 'HH24') || ':00 - ' || TO_CHAR(CheckInTime, 'HH24') ||
':59' AS HourRange, COUNT(*) AS CheckInCount
FROM CheckIn
GROUP BY TO_CHAR(CheckInTime, 'HH24')
ORDER BY CheckInCount DESC;
```

**Output:**

| HOURRANGE | CHECKINCOUNT |
|---|---|
| 08:00 - 08:59 | 11 |
| 09:00 - 09:59 | 8 |
| 10:00 - 10:59 | 7 |
| 11:00 - 11:59 | 4 |
| 12:00 - 12:59 | 3 |
| 14:00 - 14:59 | 3 |
| 07:00 - 07:59 | 2 |
| 18:00 - 18:59 | 2 |
| 17:00 - 17:59 | 2 |
| 15:00 - 15:59 | 1 |
| 19:00 - 19:59 | 1 |
| 13:00 - 13:59 | 1 |

**Query 7: Which passengers made more than three completed journeys?**

This query finds passengers that have made more than 3 completed journeys. It identifies frequent travellers.

```
SELECT co.CardID, pc.CardType, COUNT(*) AS JourneyCount
FROM CheckOut co
JOIN PassengerCard pc ON co.CardID = pc.CardID
GROUP BY co.CardID, pc.CardType
HAVING COUNT(*) > 3
ORDER BY JourneyCount DESC;
```

**Output:**

| CARDID | CARDTYPE | JOURNEYCOUNT |
|---|---|---|
| OYS001 | Oyster | 6 |
| OYS003 | Oyster | 5 |
| OYS006 | Oyster | 4 |
| OYS004 | Oyster | 4 |
| OYS005 | Oyster | 4 |
| OYS002 | Oyster | 4 |

**Query 8: Which transit stations and line changes are most popular, based on pink reader use?**

This query analyses pink reader usage. It shows popular transfer stations where line changes are made.

```
SELECT s.StationName, s.Zone, l.LineName AS ConnectedLine, COUNT(*) AS
PinkReaderUsageCount
FROM CheckIn ci
JOIN Station s ON ci.StationID = s.StationID
JOIN Line l ON ci.LineID = l.LineID
WHERE ci.CheckInType = 'PinkReader'
GROUP BY s.StationName, s.Zone, l.LineName
ORDER BY PinkReaderUsageCount DESC;
```

**Output:**

| STATIONNAME | ZONE | CONNECTEDLINE | PINKREADERUSAGECOUNT |
|---|---|---|---|
| Stratford | 2 | Central | 1 |
| Canary Wharf | 2 | DLR | 1 |
| Bank | 1 | DLR | 1 |

**Query 9: What was the most expensive journey?**

This query retrieves the details of the most expensive trip in the system.

```
SELECT co.CheckOutID, co.CardID, pc.CardType, s.StationName, l.LineName,
TO_CHAR(co.CheckOutTime, 'DD-MON-YYYY HH24:MI') AS CheckOutTime, co.FareAmount
FROM CheckOut co
JOIN PassengerCard pc ON co.CardID = pc.CardID
JOIN Station s ON co.StationID = s.StationID
JOIN Line l ON co.LineID = l.LineID
ORDER BY co.FareAmount DESC
FETCH FIRST 1 ROW ONLY;
```

**Output:**

| CHECKOUTID | CARDID | CARDTYPE | STATIONNAME | LINENAME | CHECKOUTTIME | FAREAMOUNT |
|---|---|---|---|---|---|---|
| CO036 | OYS001 | Oyster | Bank | Northern | 20-JUN-2025 09:17 | 8.6 |

**Query 10: Which station has the most check-ins and check-outs?**

This query finds the busiest station. It sorts by the total number of check-ins and check-outs.

```
SELECT s.StationID, s.StationName, s.Zone, COUNT(DISTINCT ci.CheckInID) +
COUNT(DISTINCT co.CheckOutID) AS TotalActivity
FROM Station s
LEFT JOIN CheckIn ci ON s.StationID = ci.StationID
LEFT JOIN CheckOut co ON s.StationID = co.StationID
GROUP BY s.StationID, s.StationName, s.Zone
ORDER BY TotalActivity DESC
FETCH FIRST 1 ROW ONLY;
```

**Output:**

| STATIONID | STATIONNAME | ZONE | TOTALACTIVITY |
|---|---|---|---|
| S01 | King's Cross St Pancras | 1 | 12 |

**Query 11: Which passengers have at least one incomplete journey?**

This query lists passengers with at least one incomplete journey. It identifies passengers who checked in but did not check out.

```
SELECT DISTINCT pc.CardID, pc.CardType
FROM PassengerCard pc
WHERE EXISTS (SELECT * FROM CheckIn ci
              WHERE ci.CardID = pc.CardID
              AND NOT EXISTS (SELECT * FROM CheckOut co
                              WHERE co.CardID = ci.CardID
                              AND co.CheckOutTime > ci.CheckInTime));
```

**Output:**

| CARDID | CARDTYPE |
|---|---|
| OYS002 | Oyster |
| TKT004 | Ticket |
| 45397777888889999 | BankCard |
| OYS003 | Oyster |

**Query 12: Which routes between stations are travelled most often?**

This query finds the most frequently used routes. It shows popular routes by matching check-ins and check-outs made with the same card on the same day.

```
SELECT s1.StationName || ' → ' || s2.StationName AS Route, l.LineName, COUNT(*) AS
TotalJourneys
FROM CheckIn ci
JOIN CheckOut co ON ci.CardID = co.CardID AND co.CheckOutTime > ci.CheckInTime AND
TRUNC(co.CheckOutTime) = TRUNC(ci.CheckInTime)
JOIN Station s1 ON ci.StationID = s1.StationID
JOIN Station s2 ON co.StationID = s2.StationID
JOIN Line l ON ci.LineID = l.LineID
GROUP BY s1.StationName, s2.StationName, l.LineName
HAVING COUNT(*) > 1
ORDER BY TotalJourneys DESC;
```

**Output:**

| ROUTE | LINENAME | TOTALJOURNEYS |
|---|---|---|
| Wimbledon → Paddington | District | 2 |
| Paddington → Wimbledon | District | 2 |
| Liverpool Street → Stratford | Elizabeth | 2 |